TECHNICAL PROJECT REPORT

Support Triage Pipeline

Automated Customer Ticket Classification & Routing

Author

Muhammad Haseeb

Date

May 22, 2026

Status

Implemented | All MUST + SHOULD + STRETCH requirements addressed

Technology Stack

Python 3 • Kimi k2.6 (Moonshot AI) • OpenAI-compatible SDK

This report documents the architecture, implementation, and validation of a replayable support triage pipeline built as part of a timed technical assessment. The pipeline handles all stages from raw ticket ingestion to a fully-routed agent queue.

Table of Contents

Contents

1. Executive Summary

This project implements a production-grade, replayable customer support triage pipeline that reads tickets and configuration from disk, classifies each ticket using a live LLM (Kimi k2.6 via Moonshot AI), presents predictions to a human reviewer for optional correction, and produces a final routing queue alongside a full audit trail.

All MUST and SHOULD requirements from the specification are fully implemented. Both STRETCH goals — batch resilience and LLM call logging — are also present. The pipeline runs from a clean checkout with no precomputed outputs; every artifact is regenerated at runtime.

2. Project Overview

2.1 Problem Statement

Customer support teams receive high volumes of unstructured tickets that need to be categorised, prioritised, and routed to the correct queue before an agent can action them. This pipeline automates that triage step while keeping a human review checkpoint for quality control.

2.2 Input Files

File	Purpose
`tickets.json`	Array of raw customer support tickets — may be replaced by the evaluator with any equivalent fixture
`triage_config.json`	Allowed categories, priorities, routing rules, and reply-style constraints — all enforced in code

3. Pipeline Architecture

3.1 Stage Flow Diagram

The diagram below shows every stage the pipeline transitions through, together with the artifact written at each stage. Stages are enforced sequentially in code — the final queue cannot be generated before human review is complete.

INIT	—
↓	
INPUTS_LOADED	`tickets.json` • `triage_config.json`
↓	
TICKETS_NORMALIZED	`normalized_tickets.json`
↓	
TRIAGE_PREDICTED	`triage_predictions.json` • `escalations.json` • `llm_calls.jsonl`

↓	
HUMAN_REVIEW_COMPLETE	`review_overrides.json`
↓	
FINAL_QUEUE_GENERATED	`final_queue.json` • `queue_summary.md`
↓	
RESULTS_FINALISED	`All artifacts confirmed written`

3.2 Design Principles

- Config-driven constraints — every category, priority, and route value is read from triage_config.json. The LLM cannot produce an out-of-spec value; _enforce_config() validates unconditionally.
- Normalization before LLM — normalized_tickets.json is written in pure Python before the first API call, making it deterministic and reproducible.
- Routing always re-derived — route_to and final_route_to are never copied from the model response. They are computed as routing_rules[category] in code.
- Offline capability — if no API key is configured, the deterministic keyword heuristic produces well-formed, config-valid outputs so the pipeline runs anywhere.
- Non-crashing resilience — malformed or missing predictions are caught and replaced with safe defaults (other / normal / manual_review_queue) rather than raising an exception.

4. Technical Implementation

4.1 Stage 1 — Deterministic Ticket Normalization

Each raw ticket is transformed to a normalized record in pure Python. No LLM call is made at this stage. The key fields produced are:

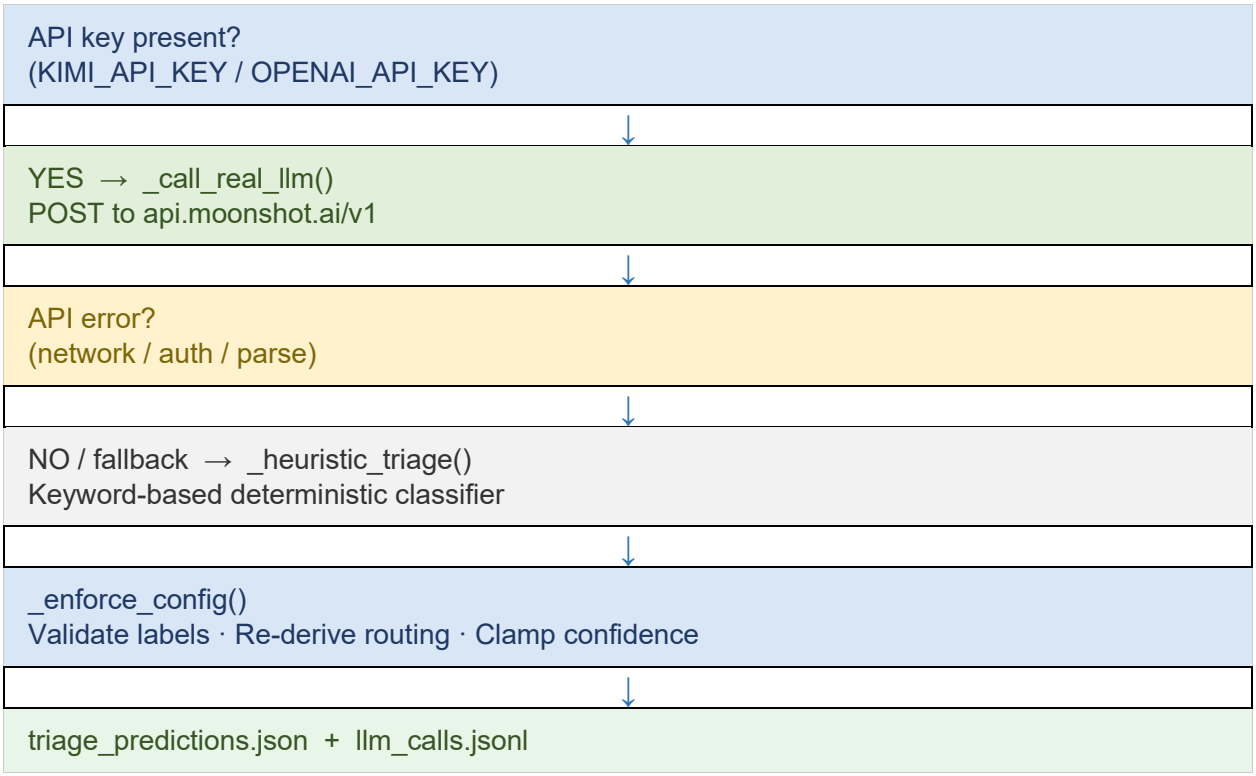
- text_for_model — built as "Subject: {subject}\nMessage: {message}"
- char_count — the exact length of text_for_model

The output is saved to normalized_tickets.json before any API call is made, ensuring the evaluator can verify the order of operations.

4.2 Stage 2 — Triage Prediction

A single prompt is constructed containing the complete triage_config.json (categories, priorities, routing rules, reply constraints) and all normalized tickets. This prompt is sent to the LLM in one call.

LLM Integration Flow



The Kimi API endpoint is OpenAI-compatible. The pipeline uses the `openai` Python SDK with a custom `base_url` pointing to Moonshot AI. No custom SDK is required.

Prompt Construction

```
You are a support-ticket triage assistant.
STRICT RULES:
  - category MUST be one of: [allowed_categories from config]
  - priority MUST be one of: [allowed_priorities from config]
  - route_to MUST be looked up from routing_rules for chosen category
  - suggested_reply tone: {tone} — max {max_words} words
  - confidence is a float in [0.0, 1.0]

CONFIG: {full triage_config.json}
TICKETS: {all normalized tickets}

Return a JSON array — one object per ticket with keys:
  ticket_id, category, priority, confidence, reason,
  suggested_reply, route_to
```

Config Enforcement (_enforce_config)

After receiving the model response, every prediction passes through this guard. Invalid category values are replaced with "other"; invalid priorities with "normal". The route_to value is always overwritten with routing_rules[category] regardless of what the model returned.

4.3 Stage 3 — Human Review Checkpoint

Before the final queue is generated, the pipeline displays a formatted table of all predictions and prompts the reviewer with the exact text specified in the requirements:

```
Enter overrides as: ticket_id,category,priority
Press Enter on an empty line when done.
```

Each override is validated against allowed_categories and allowed_priorities before being accepted. Invalid ticket IDs, categories, or priorities are rejected with an error message. The reviewer may submit zero or more overrides. All overrides are saved to review_overrides.json with the shape: ticket_id, old_category, new_category, old_priority, new_priority.

4.4 Stage 4 — Final Queue Generation

final_queue.json is generated only after human review is complete. For each ticket, the pipeline applies any recorded override and re-derives final_route_to from the routing rules. The was_overridden boolean field records whether a human correction was applied.

queue_summary.md is also produced at this stage. It contains total ticket count, category breakdown, priority breakdown, queue distribution, and a section listing any overridden tickets.

4.5 Escalation Rules (Should)

After predictions are computed, _compute_escalations() runs in deterministic Python code. A ticket is flagged for manual escalation if either condition holds:

- category == "other"
- confidence < 0.60

Results are saved to escalations.json with the ticket ID, category, confidence, and a list of escalation reasons.

4.6 LLM Call Logging (Stretch)

Every LLM invocation appends one JSON object to llm_calls.jsonl. Each entry includes:

- timestamp — ISO-8601 UTC
- stage — pipeline stage name
- mode — "real" (API called) or "simulated" (heuristic used)
- provider and model — from environment variables
- prompt_hash — SHA-256 first 16 hex characters
- prompt_chars — total prompt length
- n_inputs / n_outputs — ticket counts

5. Requirements Coverage

Tier	Requirement	Status	Notes
MUST	Deterministic Normalization	☑ Complete	Pure Python, no LLM, before any API call
MUST	Single batch LLM call	☑ Complete	One prompt for all tickets with full config
MUST	Config-enforced labels	☑ Complete	_enforce_config() guards every prediction
MUST	Routing from config only	☑ Complete	route_to always re-derived in code
MUST	Interactive Human Review	☑ Complete	Exact spec prompt, validates overrides
MUST	Final queue after review	☑ Complete	Overrides applied; routing recomputed
MUST	Queue summary markdown	☑ Complete	Category, priority, queue, overrides sections
SHOULD	Confidence field	☑ Complete	Float 0.0–1.0 on every prediction
SHOULD	Escalation rules	☑ Complete	Deterministic: other / confidence < 0.60
SHOULD	Validation script	☑ Complete	5 check groups, python validate.py
STRETCH	Batch resilience	☑ Complete	Fallback for bad predictions; LLM error catch
STRETCH	LLM call logging	☑ Complete	llm_calls.jsonl with hash, model, timestamps

6. Validation

6.1 Running the Validator

```
python validate.py
```

The validator is a standalone script with no runtime dependency on triage_pipeline.py. It can be run against any set of artifacts, including after the evaluator replaces the input files.

6.2 Validation Checks

Check	Group	What is verified
1/5	File existence & JSON validity	All 7 required artifacts are present and parse as valid JSON
2/5	Deterministic normalization	text_for_model and char_count recomputed and compared to file contents
3/5	Prediction constraints	Categories, priorities, routing, confidence range, reply word count
4/5	Override integrity	Override values are allowed; old_category / old_priority match predictions
5/5	Final queue consistency	Overrides applied correctly; routing re-derived; was_overridden flag set

6.3 Expected Output

```
============================================================
VALIDATING SUPPORT TRIAGE PIPELINE
============================================================
[1/5] Checking required files exist and parse...
  ok:   valid JSON: tickets.json
  ok:   valid JSON: triage_config.json
  ...
[2/5] Checking deterministic normalization...
  ok:   normalization deterministic for all tickets
[3/5] Checking predictions against config...
  ok:   all predictions valid
[4/5] Checking overrides...
  ok:   N override(s) validated
[5/5] Checking final queue...
  ok:   final queue consistent with predictions and overrides

============================================================
☑ VALIDATION PASSED
============================================================
```

7. Generated Artifacts

Artifact	Stage Generated	Required	Description
normalized_tickets.json	TICKETS_NORMALIZED	Required	Deterministic pre-LLM normalization
triage_predictions.json	TRIAGE_PREDICTED	Required	LLM/heuristic output, config-enforced
review_overrides.json	HUMAN_REVIEW_COMPLETE	Required	Human corrections with old/new values
final_queue.json	FINAL_QUEUE_GENERATED	Required	Post-review routing + reply
queue_summary.md	FINAL_QUEUE_GENERATED	Required	Markdown summary for agents
escalations.json	TRIAGE_PREDICTED	Should	Tickets flagged for manual escalation
llm_calls.jsonl	TRIAGE_PREDICTED	Stretch	Append-mode audit log of LLM calls

8. How to Run

8.1 Installation

```
pip install -r requirements.txt
```

requirements.txt contains: openai>=1.0.0 and python-dotenv>=1.0.0

8.2 Configuration

Set the following environment variables before running. All are optional — the pipeline degrades gracefully to the offline heuristic if none are set.

Variable	Default	Purpose
KIMI_API_KEY	—	Kimi API key (preferred)
OPENAI_API_KEY	—	Fallback if no Kimi key
LLM_BASE_URL	https://api.moonshot.ai/v1	OpenAI-compatible endpoint

LLM_MODEL	kimi-k2.6	Model name to call
LLM_PROVIDER	moonshot	Used in llm_calls.jsonl logs
LLM_TEMPERATURE	1	Sampling temperature

You may also create a .env file in the project directory — the pipeline will load it automatically via python-dotenv.

```
# .env
KIMI_API_KEY=sk-your-key-here
```

8.3 Running

```
# Interactive run (with human review prompt)
python triage_pipeline.py

# Non-interactive (skip overrides by piping empty input)
echo "" | python triage_pipeline.py

# Validate all artifacts
python validate.py
```

9. File Structure

File	Type	Role
tickets.json	INPUT	Raw support tickets (provided)
triage_config.json	INPUT	Allowed labels, routing, reply style
triage_pipeline.py	SOURCE	Main pipeline — all 7 stages
validate.py	SOURCE	Standalone end-to-end validator
requirements.txt	CONFIG	openai>=1.0.0, python-dotenv>=1.0.0
normalized_tickets.json	GENERATED	Output of Stage 2
triage_predictions.json	GENERATED	Output of Stage 3
review_overrides.json	GENERATED	Output of Stage 4
final_queue.json	GENERATED	Output of Stage 5
queue_summary.md	GENERATED	Output of Stage 5
escalations.json	GENERATED	Output of Stage 3 (Should)
llm_calls.jsonl	GENERATED	Output of Stage 3 (Stretch)

10. Key Design Decisions

10.1 Single Batch LLM Call

Rather than making one API call per ticket, all tickets are submitted in a single prompt. This is more efficient, keeps the cost lower, and produces one clean entry in the LLM call log per pipeline run. The prompt includes the full configuration so the model has all constraints in context.

10.2 Code Always Wins Over the Model

The model is used for classification reasoning and reply drafting — tasks where language understanding adds value. It is never trusted for structural decisions. Categories and priorities are validated against the config after every response. Routing is always recomputed in Python. This means the pipeline's structural outputs are guaranteed correct regardless of what the model returns.

10.3 Graceful Degradation

The pipeline has two layers of resilience. First, if the real LLM call fails (network error, invalid API key, parse error), it catches the exception and falls back to the deterministic keyword heuristic without user intervention. Second, if individual predictions are malformed or missing from the model response, they are individually replaced with safe defaults rather than crashing the run.

10.4 Reproducibility

Every artifact is regenerated from the input files on each run. Normalization is deterministic, so the same inputs always produce identical normalized_tickets.json. The human review checkpoint accepts zero overrides, making fully automated re-runs possible.

```
============================================================
STAGE: INIT
============================================================
Starting Support Triage Pipeline

============================================================
STAGE: INPUTS_LOADED
============================================================
Loaded 4 tickets from tickets.json
Loaded config from triage_config.json
  categories: ['billing_issue', 'account_access', 'product_how_to', 'bug_report', 'other']
  priorities: ['urgent', 'high', 'normal', 'low']

============================================================
STAGE: TICKETS_NORMALIZED
============================================================
Normalized 4 tickets -> normalized_tickets.json

============================================================
STAGE: TRIAGE_PREDICTED
============================================================
Built single triage prompt (sha256[:16]=b5aac09e7c1f8ee3, chars=3175)
  calling moonshot / kimi-k2.6 at https://api.moonshot.ai/v1 ...
  received 4 predictions from kimi-k2.6
Wrote 4 predictions -> triage_predictions.json
Escalations: 0 -> escalations.json

============================================================
STAGE: HUMAN_REVIEW_COMPLETE
============================================================

TICKET     CATEGORY           PRIORITY   CONF   ROUTE_TO
-------------------------------------------------------------------------
T-1001     billing_issue      high       0.98   payments_queue
T-1002     account_access     urgent     0.99   trust_and_access_queue
T-1003     product_how_to     normal     0.97   general_support_queue
T-1004     bug_report         high       0.96   technical_queue

Enter overrides as: ticket_id,category,priority
Press Enter on an empty line when done.
>
Recorded 0 override(s) -> review_overrides.json

============================================================
STAGE: FINAL_QUEUE_GENERATED
============================================================
Final queue: 4 entries -> final_queue.json
Wrote queue summary -> queue_summary.md

============================================================
STAGE: RESULTS_FINALISED
============================================================
All artifacts written:
  - normalized_tickets.json
  - triage_predictions.json
  - review_overrides.json
  - final_queue.json
  - queue_summary.md
  - escalations.json
  - llm_calls.jsonl
```

End of Report — Muhammad Haseeb — May 2026